LÖSUNGEN · LEHREREXEMPLAR

W3·1 – Rekursion: Lösungen

Zum Arbeitsblatt `AQ_W3_AB1_Rekursion.docx` · IDE-Level 3.1: siehe `AQ_W3_L3_Labyrinth_LSG.txt` (Klasse Rekursion) · Fraktale: `AQ_W3_Fraktale_LSG.txt` · alle Werte, Ausgaben und Fehlermeldungen mit Java (javac 21) nachgerechnet · Rekursionstiefe: Erstaufruf = 0

Teil A

Aufgabe 0 – Matroschka, Spiegel und Vorfahren

a) „Öffne die Puppe. Ist sie massiv, ist die Anzahl 1 (**Basisfall**). Sonst ist die Anzahl 1 + die Anzahl der Puppen in der inneren Puppe (**Rekursionsschritt**, dasselbe Verfahren für ein kleineres Problem).“ Ohne kleinste Puppe würde man unendlich weiter öffnen – das Verfahren endet nie (wie eine Methode ohne Basisfall). **b)** Jedes Spiegelbild ist kleiner und dunkler; irgendwann ist es kleiner als ein Bildpunkt bzw. das Licht zu schwach – die Wirklichkeit hat einen „Basisfall“. **c)** „Ein Vorfahre von X ist ein Elternteil von X **oder** ein Vorfahre eines Elternteils von X.“ Die Definition benutzt sich selbst und endet bei Personen mit unbekannten Eltern.

Aufgabe 1 – Das Zettel-Spiel im Spiegelsaal

Rekursionstiefe	Aufruf	n	Rückgabe
0	spiegel(3)	3	3 + "21|12" + 3 = "321|123"
1	spiegel(2)	2	2 + "1|1" + 2 = "21|12"
2	spiegel(1)	1	1 + "|" + 1 = "1|1"
3	spiegel(0)	0	"|" (Basisfall)

a) Rückgabewert **"321|123"**. **b)** 4 Personen (Aufrufe) für spiegel(3), allgemein **n + 1** (für spiegel(5): 6); größte Rekursionstiefe **3** = n. **c)** Für n ≥ 1 liefert die Änderung dasselbe (spiegel(3) = "321|123", sogar mit nur 3 Aufrufen). Aber `spiegel(0)` ruft spiegel(−1), spiegel(−2), … auf – der Basisfall n = 1 wird **übersprungen** → **StackOverflowError**. Der Basisfall muss den kleinsten erlaubten Fall abdecken.

Aufgabe 2 – Rekursionstabellen im Gewölbe

Rekursionstiefe	Aufruf	a	Rückgabe
0	rest(23, 5)	23	3
1	rest(18, 5)	18	3
2	rest(13, 5)	13	3
3	rest(8, 5)	8	3
4	rest(3, 5)	3	3 (Basisfall a < b)

b) rest(a, b) liefert den **Rest der ganzzahligen Division** a mod b (`a % b`): b wird so lange abgezogen, bis weniger als b übrig ist. rest(100, 7) = 2 mit **15 Aufrufen** (14-mal abziehen + Basisfall; allgemein a / b + 1). In der Spalte Rückgabe steht **überall dieselbe Zahl**: Das Ergebnis des Basisfalls wird nur unverändert durchgereicht (**Endrekursion** – nach dem Selbstaufruf wird nicht mehr gerechnet).

Rekursionstiefe	Aufruf	n	Rückgabe
0	geheim(4)	4	9 + 7 = 16
1	geheim(3)	3	4 + 5 = 9
2	geheim(2)	2	1 + 3 = 4
3	geheim(1)	1	0 + 1 = 1

Rekursionstiefe	Aufruf	n	Rückgabe
4	geheim(0)	0	0 (Basisfall)

c) geheim(4) = **16**. **d)** Werte für n = 0 … 7: 0, 1, 4, 9, 16, 25, 36, 49 → geheim(n) = **n^2**. Begründung: Gilt geheim(n − 1) = $(n-1)^2$, dann liefert der Rekursionsschritt $(n-1)^2 + 2n - 1 = n^2 - 2n + 1 + 2n - 1 = n^2$; Start: geheim(0) = 0 = 0^2. (Die Quadratzahlen sind die Summen der ersten n ungeraden Zahlen.)

Aufgabe 3 – Hin- und Rückweg mit Fackeln

a) `an 3` · `an 2` · `an 1` · `aus 1` · `aus 2` · `aus 3`. **b)** `an 1` · `an 2` · `an 3`. In fackeln2 steht die Ausgabe **hinter** dem Selbstaufruf: fackeln2(3) wartet auf fackeln2(2), dieses auf fackeln2(1), dieses auf fackeln2(0). Erst der oberste Rahmen (n = 1) wird zuerst fertig und gibt aus; danach die Rahmen darunter. Ausgaben vor dem Aufruf erscheinen auf dem **Hinweg** (absteigend), Ausgaben danach auf dem **Rückweg** (aufsteigend). **c)** Der Basisfall ist n ≤ 0: Die Bedingung `n > 0` ist falsch, die Methode tut nichts und endet (kein sonst-Zweig nötig). Jeder Selbstaufruf verringert n um 1, also wird nach n Schritten n = 0 erreicht.

Aufgabe 4 – Bug-Hunt: Wenn der Stapel überläuft (10 BE)

	Aufruf	Ergebnis bzw. Abbruch (javac/java)	Fehler und Korrektur
A	produkt(3)	StackOverflowError	Der Selbstaufruf steht **vor** der Basisfall-Abfrage – sie wird nie erreicht (auch produkt(1) läuft endlos). Korrektur: `if (n == 1) { return 1; }` als erste Anweisung, dann `return n * produkt(n - 1);` → 6
B	laenge("abc")	4	Falscher Wert im Basisfall: Die leere Zeichenkette hat die Länge 0. Korrektur: `return 0;` → 3
C	umdrehen("Gold")	"dloG" (richtig)	Basisfall deckt die leere Zeichenkette nicht ab: Für "" wird `"".substring(1)` ausgeführt.
C	umdrehen("")	StringIndexOutOfBoundsException	Korrektur: `if (s.length() <= 1)` → "" liefert ""
D	anzahlX("xaxx")	1	Das Ergebnis des Selbstaufrufs wird **verworfen**; gezählt wird nur das erste Zeichen. Korrektur: `int rest = anzahlX(s.substring(1));` und `return 1 + rest;` bzw. `return rest;` → 3

a) je Zeile ca. 1 BE (4 BE), **b)** je Methode 1 BE. Alle korrigierten Fassungen mit Java geprüft: produkt(3) = 6, laenge("abc") = 3, laenge("") = 0, umdrehen("") = "", anzahlX("xaxx") = 3.

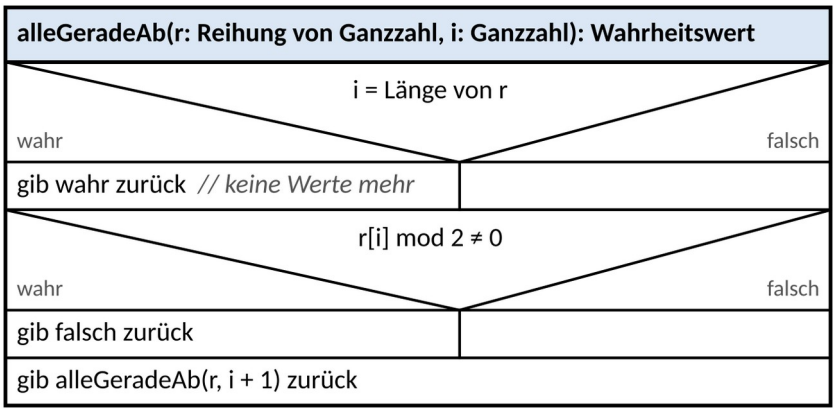
c) produkt(3) legt einen Rahmen an und ruft sofort produkt(2) auf, dieses produkt(1), produkt(0), produkt(−1), … Kein Rahmen kommt je bis zur Zeile `if (n == 1)`, weil jeder zuerst auf seinen Selbstaufruf wartet. Der Stapel wächst, bis der Speicher erschöpft ist → StackOverflowError. Der Basisfall ist zwar vorhanden, wird aber **erst nach** dem Selbstaufruf geprüft (2 BE).

Aufgabe 5 – Schleifen-Bann (6 BE)

```
public static String sterne(int n) {
    if (n == 0) {
        return "";                      // Basisfall
    }
    return "*" + sterne(n - 1);          // Rekursionsschritt
}

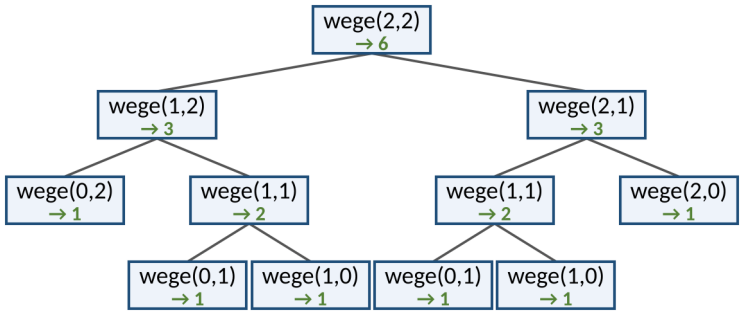
public static boolean alleGerade(int[] r) {
    return alleGeradeAb(r, 0);           // Huellmethode
}

public static boolean alleGeradeAb(int[] r, int i) {
    if (i == r.length) {
        return true;                    // keine Werte mehr: alle waren gerade
    }
    if (r[i] % 2 != 0) {
        return false;                   // ungerader Wert gefunden
    }
    return alleGeradeAb(r, i + 1);
}
```



Mit Java geprüft: sterne(4) = "****", **sterne(0) = ""; alleGerade({4, 8, 2}) = true, alleGerade({4, 7, 2, 6}) = false, leere Reihung: true (alle Ergebnisse gleich der Schleifenfassung).** d) **{4, 8, 2}:** 4 Aufrufe **(i = 0 bis 3, Basisfall i = Länge); {4, 7, 2, 6}:** 2 Aufrufe** (Abbruch bei 7). Häufige Fehler: Basisfall `i == r.length - 1` (leere Reihung stürzt ab), `return alleGeradeAb(r, i)` ohne + 1 (StackOverflowError).

Aufgabe 6 – Die Wege durchs Gitterverlies (8 BE)



a) wege(2, 2) = **6** (3 BE). **b) 11 Aufrufe** (Knoten des Baums), größte Rekursionstiefe **3** (z. B. wege(2,2) → wege(1,2) → wege(1,1) → wege(0,1)); wege(1, 1) wird **2-mal** aufgerufen (2 BE).

n	1	2	3	4	5	6
wege(n, n)	2	6	20	70	252	924
Aufrufe	3	11	39	139	503	1847

n	1	2	3	4	5	6
größte Tiefe	1	3	5	7	9	11

c) Die Aufrufzahl ist A = 2 · wege(n, n) − 1 (jeder Basisfall liefert 1, also gibt es wege(n, n) Blätter; jeder innere Knoten hat zwei Kinder). Sie wächst je Stufe etwa um den Faktor 3,6. wege(10, 10) = 184756 braucht **369.511 Aufrufe**, weil dieselben Teilprobleme immer wieder neu berechnet werden (wege(1, 1) bei wege(3, 3) schon 6-mal). Besser: eine **2D-Reihung** w[z][s] zeilenweise füllen – w[z][0] = w[0][s] = 1, sonst w[z][s] = w[z − 1][s] + w[z][s − 1]. Das sind nur 11 · 11 = 121 Rechenschritte (Ausblick: „dynamische Programmierung“). Alternativ: Ergebnis = Binomialkoeffizient (z + s über z) (3 BE).

Aufgabe 7 – Level 3.1

Musterlösungen und häufige Fehler: `AQ_W3_L3_Labyrinth_LSG.txt`, Klasse Rekursion und Abschnitt „HÄUFIGE SCHÜLERFEHLER – Level 3.1“. **c)** dual(13) = **"1101"**, fibonacci(6) = **8**, Aufrufe: **25** (Aufrufe für n = 0 … 10: 1, 1, 3, 5, 9, 15, 25, 41, 67, 109, 177; fibonacci(30) braucht bereits 2.692.537 Aufrufe). Typische Abweichung: 7 statt 15 bei fibonacci(5), wenn `aufrufe++` erst hinter dem Basisfall steht. Code-Review: Schleifen in den REKURSIV-Methoden sind per Mutationstest grün – deshalb Sichtprüfung.

Aufgabe 8 – Deko-Werkstatt

Tiefe n	0	1	2	3	4	Formel für n
Aufrufe baum	1	3	7	15	31	2^(n+1) − 1
Striche baum	0	1	3	7	15	2^n − 1

Deko	k	Aufrufe Tiefe 2	Tiefe 3	Tiefe 4	Wachstum je Stufe
baum3	3	13	40	121	≈ ·3, A(n) = (3^(n+1) − 1) / 2
koch	4	21	85	341	≈ ·4, A(n) = (4^(n+1) − 1) / 3
sierpinski	3	13	40	121	≈ ·3 (Striche: 3^(n+1))

c) Jeder Aufruf mit tiefe > 0 löst k Aufrufe mit tiefe − 1 aus: A(0) = 1, A(n) = 1 + k · A(n − 1). Steigt die Tiefe um 1, vervielfacht sich die Zahl der Aufrufe also etwa um k (exponentielles Wachstum). **d)** H-Baum: Striche = 2^(n+1) − 2 (n = 0 … 4: 0, 2, 6, 14, 30). Alle Werte stimmen mit der Kennzahlen-Tabelle in `AQ_W3_Fraktale_LSG.txt` überein (dort durch Ausführen in der IDE ermittelt; hier zusätzlich mit einem Java-Zählmodell nachgerechnet). Typischer Fehler: `aufrufe++` hinter dem Basisfall → Baum Tiefe 3: 7 statt 15.

Für Schnelle

Hanoi: Züge für n = 1 … 10: 1, 3, 7, 15, 31, 63, 127, 255, 511, 1023 = 2^n − 1. Für n = 3: A->C, A->B, C->B, A->C, B->A, B->C, A->C. 64 Scheiben: 2^64 − 1 Züge ≈ **5,845 · 10^11 Jahre** – kein Grund zur Eile. **Euklid:** ggT(84, 36) → ggT(36, 12) → ggT(12, 0) = **12** (Tiefen 0, 1, 2); b wird echt kleiner (a % b < b) und bleibt ≥ 0, also wird b = 0 erreicht. **Spiegelwort:** `if (s.length() == 0) { return ""; } return s.charAt(0) + spiegelwort(s.substring(1)) + s.charAt(0);` liefert für "abc" "abccba".